Hola Brian, David y Federico.

Estuve revisando en detalle la segunda entrega completa de TechFlow: README, propuesta original, caso de reparación, módulos P0, arquitectura, DER y esquema PostgreSQL.

En términos generales considero que **Modelo y Arquitectura están bien encaminados y la entrega puede darse por conforme con algunas correcciones importantes**.

Quiero destacar primero que hubo una mejora clara respecto de la primera propuesta.

Recortaron correctamente el alcance alrededor del circuito:

Cliente → Equipo → Orden → Diagnóstico → Presupuesto → decisión → Reparación → Listo → Entrega

y dejaron proveedores/compras, analítica avanzada e IA para etapas posteriores.

También considero correctas la separación entre estado técnico y estado financiero, la distinción entre repuesto reservado y utilizado, la elección de PostgreSQL y la decisión de no persistir PIN/contraseñas de los equipos.

Es decir, varias observaciones de la primera devolución fueron incorporadas correctamente.

Hay, sin embargo, un primer punto metodológico que quiero corregir.

Para esta instancia les había pedido elegir uno de los talleres relevados y describir un caso completo. En cambio, construyeron “TecnoRepair” combinando características de los tres perfiles.

El escenario compuesto es útil para probar el modelo, pero **no sirve para afirmar que una regla fue validada por un taller real**.

Por ejemplo, “no se entrega un equipo con saldo pendiente”, “a los 30 días se realiza un segundo aviso” o “el repuesto se reserva al presupuestar” deben quedar diferenciadas entre:

regla observada/confirmada por un negocio real

y

decisión adoptada por el grupo para el MVP.

Con un taller concreto, aunque quede **anonimizado**, es suficiente.

El principal problema conceptual que encontré está en el propio caso completo.

Ustedes plantean una situación donde dentro de la misma Orden pueden coexistir:

conector → trabajo aprobado y en reparación

pantalla → esperando repuesto

altavoz → esperando decisión del cliente.

Sin embargo, el modelo actual tiene un único ordenes.estado.

Una Orden no puede estar simultáneamente EN_REPARACION, ESPERANDO_REPUESTO y ESPERANDO_APROBACION_ADICIONAL.

Además presupuesto_items sólo distingue pendiente/aprobado/rechazado; tampoco puede representar el estado técnico independiente de cada trabajo.

Necesitan revisar esta parte del dominio.

Una posibilidad bastante natural sería que una Orden posea varios **trabajos/ítems de reparación**, y que cada uno conserve su falla/hallazgo, diagnóstico, decisión del cliente, estado técnico y repuestos.

La Orden puede mantener un estado global más sencillo, mientras cada trabajo explica lo que ocurre internamente.

Esto permitiría representar correctamente el escenario que ustedes mismos diseñaron.

Relacionado con esto, definan exactamente qué significan presupuesto v1, v2, v3.

En el documento aparecen como presupuestos adicionales: v1 contiene el conector, v2 la pantalla y v3 el altavoz.

Pero si son realmente “versiones”, normalmente una nueva versión representa el presupuesto completo que reemplaza a la anterior.

Esta diferencia importa mucho porque luego dicen que el saldo se calcula usando el “presupuesto vigente”.

Si v3 sólo contiene el altavoz, no representa todo lo que el cliente debe por conector + pantalla.

Decidan una única semántica:

versiones completas

o

presupuestos/cotizaciones adicionales independientes.

No mezclen ambos conceptos.

También falta información para reconstruir un presupuesto.

PRESUPUESTO_ITEM almacena descripción, repuesto y mano de obra, pero no conserva precio del repuesto ni cantidad. PRESUPUESTO.total queda como un número independiente.

El sistema debería poder calcular y explicar:

qué se presupuestó, cuánto costaba cada componente y cómo se obtuvo el total.

Conserven el precio aplicado como snapshot, porque un cambio futuro en el catálogo de repuestos no debe modificar un presupuesto histórico.

Hay otra corrección importante en EQUIPO.

Actualmente estado_fisico_ingreso y accesorios_entregados pertenecen al equipo.

Pero ustedes mismos establecen que un mismo equipo puede volver varias veces al taller.

El estado físico y los accesorios corresponden a **cada ingreso/orden**, no al dispositivo permanentemente.

Una notebook puede ingresar hoy con cargador y seis meses después sin cargador y con otros daños.

Muevan esos datos al ingreso/orden y dejen en Equipo únicamente los datos relativamente estables.

Revisaría también IMEI como único identificador específico, porque el sistema pretende manejar celulares, notebooks, consolas y otros dispositivos. Puede ser conveniente utilizar un identificador/serie genérico y mantener IMEI como dato opcional cuando corresponda.

Otro punto central es la **trazabilidad**.

El principal problema que TechFlow pretende resolver es justamente poder reconstruir qué ocurrió con una reparación.

Sin embargo la Orden conserva solamente su estado actual.

Incorporaría un historial sencillo de cambios de estado con:

estado anterior, estado nuevo, fecha, usuario y motivo/comentario cuando corresponda.

No necesitan una arquitectura compleja de eventos. Pero una Orden que hoy está LISTO debería permitirnos saber cuándo estuvo diagnosticada, cuándo esperó aprobación, cuándo comenzó la reparación y quién produjo cada cambio.

También necesitamos recuperar otra pregunta que habíamos marcado en la primera devolución:

¿quién está atendiendo este equipo?

Actualmente el esquema no posee usuarios/técnicos ni un responsable de Orden.

Entiendo que quieran postergar una matriz completa Administrador/Técnico/Recepción, pero al menos necesitamos una identidad de usuario y un técnico/responsable asociado a la reparación.

Esto además debe coincidir con la arquitectura, porque actualmente declaran OAuth2 + JWT pero no existe una entidad de usuario en el modelo.

Respecto de stock, la separación reservado/utilizado es correcta, pero completen el ciclo.

Definan cantidad reservada, disponibilidad efectiva y qué ocurre cuando el presupuesto es rechazado o queda sin respuesta.

Recomendaría hablar en una reserva de estados como:

RESERVADA → UTILIZADA

o

RESERVADA → LIBERADA.

“Disponible” es una propiedad del stock, no demasiado clara como estado histórico de una reserva.

Además, cuando un repuesto se utiliza, el cambio de reserva y la disminución del stock deben realizarse dentro de la misma transacción.

Y quiero una prueba específica:

stock disponible = 1 → dos órdenes intentan reservar simultáneamente → una sola puede hacerlo.

Revisen también ordenes.repuesto_sugerido_id. Una Orden puede necesitar varios repuestos y los ítems de presupuesto ya poseen relación con repuestos, por lo que ese único campo en la Orden puede quedar obsoleto o contradictorio.

En la máquina de estados falta además contemplar el caso donde el trabajo no termina en reparación.

¿Qué ocurre si el cliente rechaza todo el presupuesto o el equipo se declara irreparable?

La Orden igualmente deberá terminar devolviéndose al cliente. Definan cómo representa el sistema ese cierre sin forzarla a pasar por un estado de reparación.

Respecto de los pagos, está bien que existan como entidad independiente y que el estado financiero no se mezcle con el técnico.

Pero estado_pago no debería quedar librado a una edición manual independiente de los pagos. La suma de pagos y el total aprobado deben determinar si está pendiente, parcial o pagado.

Además protejan al menos:

monto > 0

y la regla:

una Orden no puede pasar a ENTREGADO si conserva saldo pendiente.

Esa regla debe estar implementada y testeada en backend.

También ajustaría la afirmación sobre margen por reparación. Actualmente el esquema no conserva el costo histórico de los repuestos ni define el costo real de mano de obra, por lo que todavía no podemos calcular un margen fiable. Como métricas quedaron fuera de P0, no ampliaría ahora el proyecto por esto: simplemente dejen esa métrica para cuando exista una política de costos correctamente definida.

No olviden además uno de los problemas originales del relevamiento: las consultas repetitivas por WhatsApp.

El caso habla de “mensaje preparado”, pero esa funcionalidad no aparece todavía en el núcleo. No necesitan integrar WhatsApp. Puede ser simplemente un botón para generar/copiar un mensaje de estado o, más adelante, un enlace seguro de consulta de la orden.

Finalmente, hay un punto importante respecto de la etapa académica.

Tanto backend/README.md como frontend/README.md indican que la codificación comenzará una vez aprobada la segunda entrega.

Quiero corregir ese criterio.

La etapa de Modelo y Arquitectura **no implica terminar completamente el diseño antes de comenzar a programar**.

El enfoque que buscamos es:

modelar → implementar un recorrido pequeño → comprobarlo → corregir el modelo → continuar.

De hecho, el escenario que desarrollaron ya nos permitió descubrir que el modelo actual no representa correctamente trabajos paralelos dentro de una misma Orden. El código permitirá descubrir otras situaciones similares.

Estamos ya en octubre, por lo que no esperen otra aprobación para comenzar.

El próximo avance debería incluir como mínimo un vertical ejecutable:

cliente → equipo → orden → diagnóstico → trabajo/presupuesto → aprobación → reserva de repuesto → reparación → listo → pago → entrega.

Luego prueben una segunda situación:

durante la reparación aparece una nueva falla y se genera un trabajo adicional sin alterar lo que ya estaba aprobado.

No comiencen todavía por IA, compras, dashboard ni una interfaz sofisticada.

Primero demuestren que TechFlow puede conservar correctamente la historia de una reparación real.

En síntesis:

Modelo y Arquitectura: conforme con correcciones.

La evolución respecto de la primera propuesta es positiva. El siguiente salto de calidad no está en sumar más módulos sino en hacer que el modelo represente fielmente el escenario que ustedes mismos construyeron y comenzar inmediatamente a validarlo mediante software.

Saludos,
Oscar